

Last updated on **Apr 30, 2026**

Lakebase Provisioned

! INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

Lakebase is a fully managed Postgres online transaction processing (OLTP) database engine integrated into the Databricks Data Intelligence Platform. Lakebase lets you create and manage OLTP databases stored in Databricks-managed storage, integrating with your Lakehouse for real-time transactional workloads.

Learn about Lakebase Provisioned

Understand Lakebase architecture, capabilities, and limitations.

Topic	Description
What is Lakebase?	Overview of Lakebase architecture, use cases, and Databricks integration.
Database instance architecture	Learn about database instance components, capabilities, and limitations.
Database role types and permissions	Understand the different Postgres roles and their privileges in Lakebase.

 Ask Genie

Topic	Description
PostgreSQL compatibility	Learn about PostgreSQL compatibility, limitations, and optimization tips.

Get started with Lakebase Provisioned

For database owners and admins

If you're setting up a new Lakebase database for your team:

Task	Description
Create a database instance	Set up your first Lakebase Provisioned database.
Add users and set permissions	Give other users access to your database and control what they can do.

For database users

If you need to access an existing Lakebase database:

Task	Description
Connect to your database	Get the credentials you need to access your Lakebase database.
Query your data	Use various tools to query your PostgreSQL data including SQL editor, notebooks, and psql client.

Data integration and synchronization Ask Genie

Connect Lakebase with your existing Databricks data and workflows.

Topic	Description
Register with Unity Catalog	Optionally register your PostgreSQL database as a catalog in Unity Catalog for federated queries.
Serve lakehouse data with synced tables	Create synced tables to serve Unity Catalog data through your Lakebase database instance for operational applications.

Advanced features

Explore advanced capabilities for production workloads and enterprise use cases.

Topic	Description
Restore data and time travel	Use child instances to perform time travel and restore data to specific points in time for data recovery, compliance auditing, and development environments.
Monitoring and observability	Monitor your database performance and health using built-in metrics and logging.

Advanced configuration

Explore advanced capabilities for production workloads and enterprise use cases.

Topic	Description
High availability	Configure high availability for your database instances to ensure business continuity.
 Ask Genie	

Topic	Description
Restore window	Set the restore window for your database instance for point-in-time recovery.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Mar 27, 2026**

What is Lakebase Provisioned?

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page introduces Lakebase Provisioned, a fully managed Postgres OLTP database engine, integrated into the Databricks Data Intelligence Platform. A database instance is a type of Databricks compute that provides the storage and compute for running a Postgres server that manages multiple databases.

Overview

An online transaction processing (OLTP) database is a specialized type of database system designed to efficiently handle high volumes of real-time transactional data. Lakebase allows you to create an OLTP database on Databricks, and integrate OLTP workloads with your Lakehouse. This OLTP database enables you to create and manage databases stored in Databricks-managed storage.

Using an OLTP database in conjunction with the Databricks platform significantly reduces application complexity. Lakebase is well integrated with [Databricks Feature Store](#), [SQL warehouses](#), and [Databricks Apps](#). Using sync tables provides a simple and performant way to sync data between OLTP and online analytical processing (OLAP) workloads.

NOTE

This page describes Lakebase Provisioned. For the latest version of Lakebase, which supports autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

Region availability

Supported regions:

- us-east-1
- us-east-2
- us-west-2
- eu-central-1
- eu-west-1
- ap-south-1
- ap-southeast-1
- ap-southeast-2

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Mar 26, 2026**

What is a database instance?

❗ INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

A Lakebase database instance manages storage and compute resources and provides the endpoints that users connect to. This page includes an overview that describes a database instance and its general limitations.

Database instance architecture

A database instance consists of the following components:

- **Database instance:** Acts as the central object managing storage and compute resources. It provides one or more endpoints for user connections.
- **Compute nodes:** Represents a group of compute nodes, where one node is the *primary*, handling write operations, and the others are *secondary*. Secondary compute nodes are optional. They are separated into different zones to provide high availability. Compute nodes can be paused, retaining storage while halting compute resources.
- **Endpoints:** The default database endpoint routes read and write requests to the primary node. Read-only endpoints are optional. They route requests to secondary nodes, which can help to balance read and write-heavy workloads.
- **Failure handling:** If the primary node fails and your instance is configured for [high availability](#), a secondary node automatically takes over as the primary.

 Ask Genie

system availability.

Key capabilities

Feature	Description
Database instance creation	Learn how to create a database instance, start and stop it, and delete it.
Manage instance capacity	Scale your database instance capacity to fit your workload needs.
High availability	Configure your database instance for high availability in case the primary compute is unavailable.
Restore data and time travel	Use child instances to restore your database to a specific point in time and perform time travel operations.
Monitor database instance	Monitor the performance metrics of your database instance to optimize performance.

Access a database instance

Access and manage a database instance using the UI or API. For detailed setup and connection instructions, see [Create a database instance](#). For a complete list of available operations, see the [REST API reference](#).

To access a database instance in the UI:

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Click the instance to open the instance details page.

Limitations and considerations

- A workspace allows a maximum of ten instances.
- Each instance supports up to 1000 concurrent connections.
- The logical size limit across all databases in an instance is 2 TB.
- Database instances are scoped to a single workspace and don't support cross-workspace access. Users are able to see these tables in Catalog Explorer if they have the required Unity Catalog permissions from other workspaces attached to the same metastore, but they cannot access the table contents.
- Databricks recommends configuring [Private Link](#) as a security best practice. Lakebase Provisioned requires only standard inbound Private Link. Inbound Private Link for performance-intensive services is not needed.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 15, 2026**

Create and manage a database instance

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

To get started with OLTP workloads, create a Lakebase Provisioned database instance using the Databricks UI, API call, Python SDK, or CLI.

Create a database instance

Create a database instance with recommended defaults. You need to provide an instance name (1–63 characters, letters and hyphens only). As the creator, you are the database owner with the `databricks_superuser` role.

Most workspace users can create database instances by default. If you encounter permission issues, see [database instance permissions](#).

UI Python SDK CLI curl

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.

 Ask Genie

3. Click **Create database instance**.
4. Configure your instance:
 - **Name**: Enter a database instance name (1–63 characters, letters and hyphens only).
 - **Capacity**: Select the compute size (default: 2 CU). See [Instance size](#).
 - **Serverless usage policy**: Select a budget policy to attribute serverless usage and billing (optional). See [Serverless usage policies](#).
5. (Optional) Expand **Advanced Settings** to configure:
 - **Create from parent**: Create a copy-on-write clone from an existing instance, including data up to a specific point in time. See [Create from parent](#).
 - **Enable HA**: Enable high availability with failover nodes, configure readable secondaries, and set the number of HA nodes. See [High availability](#).
6. Click **Create**.

Advanced settings

You can also configure these features during creation or after creation by editing your instance:

Feature	Description
Serverless usage policy	Select a budget policy for your database instance to attribute serverless usage and billing to specific budgets. You can also add custom tags .
Instance size	Scale compute resources for your workload performance requirements (defaults to 2 CU).
Restore window	Set the retention window (2–35 days, default 7 days) for point-in-time recovery.
High availability	Add failover nodes to ensure business continuity for production workloads.
Create from parent	Create a copy-on-write clone from an existing database instance.  Ask Genie

Stop or start an instance

To stop or start a database instance, you must have `CAN MANAGE` permissions on it. To stop or start an instance, use the Databricks UI, API call, Python SDK, or CLI.

UI Python SDK CLI curl

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Click the database instance you want to stop or start.
4. Click **Stop** or **Start** in the upper-right corner of the page.

Behavior when stopped

Database instance behavior:

- Data is preserved.
- The instance cannot be used for read or write operations.
- [Synced tables](#) do not serve reads.
- Lakeflow Spark Declarative Pipelines (LDP) do not detect stopped instances and might return errors.
- Registered catalogs on stopped instances do not show schema details in the UI.

Functional limitations:

- You cannot create or delete `DatabaseTables` or `DatabaseCatalogs`.
- You can delete or resize a stopped instance. Capacity changes take effect when the instance restarts.
- You can stop pipelines.

Behavior when started

- The instance enters the `STARTING` state and becomes `AVAILABLE` when ready.

Limitations

- LDP do not detect stopped instances and can return errors.
- Registered catalogs on stopped instances do not show schema details in the UI.

Delete an instance

Be cautious when deleting the database instance, as doing so will delete all associated data.

You must have `CAN MANAGE` permissions on the database instance. If you are not the owner of the tables or catalogs, you must reassign ownership to yourself. Workspace admins can delete database instances they do not own.

Databricks recommends deleting all associated Unity Catalog catalogs, synced tables, and child instances before deleting the database instance. Otherwise, attempting to view catalogs or run SQL queries that reference them results in errors.

UI Python SDK CLI curl

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to delete.
4. Select **Catalogs** in the Lakebae App sidebar to view the full list of database catalogs associated with the database instance.
5. For each database catalog, [delete all synced tables](#), including those that are located in managed catalogs and are not registered as database catalogs.
6. Click  > **Delete catalog**.

Update a serverless usage policy of a database instance

Serverless usage policies consist of tags that are applied to any serverless compute activity incurred by a user assigned to the policy. By tagging a database instance with a serverless usage policy, you can attribute billing and usage costs to particular

policies, making it easier to track, manage, and control spend across serverless resources.

Use the UI to update the budget policy of a database instance:

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to update the billing policy for.
4. Click **Edit** in the upper-right.
5. Select a **Serverless usage policy**.
6. Click **Save**.

Next steps

- [Serve lakehouse data with synced tables](#).
- [Connect and query your database instance from external tools, the SQL editor, or a notebook](#).
- [Register database instance in Unity Catalog](#).
- Allow other users access to the database instance from Databricks. See [Manage permissions](#) and [Manage PostgreSQL roles](#).

Limitations and requirements

The following sections describe limits and configuration requirements for managed database instances. For constraints on instance creation and usage, see [Limitations and considerations](#).

Instance name requirements

- Must be 1 to 63 characters long.
- Must begin with a letter.
- Can contain only alphanumeric characters and hyphens.
- Cannot include two consecutive hyphens.

Is this page

as this page helpful?

☐ Yes	☐ No
☐ Send feedback	

Last updated on **Mar 11, 2026**

Manage instance capacity

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page explains the options for right-sizing your Lakebase instance capacity and how to manage it.

Change instance capacity

To resize an instance, you must have `CAN MANAGE` permissions. Resizing can take several minutes. The capacity change takes effect when the instance is restarted.

UI [curl](#) [Python SDK](#) [CLI](#)

1. Click **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to change the capacity for.
4. Click **Edit** in the upper-right corner.
5. Use the **Capacity** drop-down menu to select the new instance size.
6. Click **Save**.

Best practices

Each capacity unit allocates about 16GB of RAM to the database instance, along with all associated CPU and local SSD resources. Scaling up increases these resources linearly. Postgres distributes the allocated memory across multiple components:

- Database caches
- Worker memory
- Other processes with fixed memory requirements

Performance varies based on data size and query complexity.

Before scaling, test and [optimize queries](#). Storage scales automatically.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Mar 11, 2026**

Configure for high availability

This page describes how to configure a Lakebase database instance for high availability by enabling readable secondary instances.

❗ INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

To enable high availability, specify additional nodes as part of a database instance. If the primary compute becomes unhealthy or unavailable, a high availability node is utilized to perform failover, and the secondary node is promoted to primary.

You can also enable readable secondaries, where the high availability nodes can serve read-only workloads using a separate DNS endpoint (`instance-ro-{uuid}` compared with `instance-{uuid}`).

Enable database instance for high availability

If you set the number of high availability nodes to one, high availability and readable secondaries are disabled. Otherwise, you have one primary node, and the rest are high-availability nodes. The maximum number of high-availability nodes is three per database instance.

Specify the number of high-availability nodes when creating your database instance. See [Create a database instance](#).

Perform the following steps to modify the database instance using the UI or API.

UI **curl** **Python SDK** **CLI**

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to update.
4. Click **Edit** in the upper-right corner.
5. Under the Advanced settings drop-down, select **Enable HA HA pool node size (including primary)**.
6. Select **Allow readable secondaries** and choose the number of **HA nodes**.
7. Click **Save**.

Compute recovery

With high availability nodes configured, your database instance's primary node is protected by high availability. If the primary node becomes unavailable, the database instance automatically fails it over to a secondary node, promoting it as the new primary node. As compute nodes are stateless, failures don't affect your data, and your connection string remains unchanged. The unavailability is resolved in seconds to minutes, depending on the type of failure. During a failover, active connections are dropped, so your application should be configured to handle these brief disconnections and reconnect automatically.

Secondary nodes in your database instance are also auto-recovered in minutes when issues occur. If you enable readable secondaries, Databricks recommends that you have at least three high-availability nodes, in case the primary failover affects the read-only connection. Your application still needs a reconnection mechanism to handle the brief downtime.

Execute read-only queries from Databricks SQL Editor

To connect to your readable secondaries and run read-only queries from the Databricks SQL editor, see [Access a database instance from the SQL editor](#).

Limitations

- Performance takes time to recover. Queries might initially run more slowly until they are rebuilt because the new primary node has no session-specific data and a local cache for frequently accessed data.
- Cross-region replication is not supported. In the event of a region-wide outage, the availability depends on the cloud provider restoring service to the affected region.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 15, 2026**

Restore data and time travel

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page explains how to use child instances to restore data and perform time travel on your Lakebase database instance.

When should you create a child instance?

Creating a child instance is useful in the following scenarios:

- **Accidental data loss recovery:** If a critical table was dropped at 10:23 AM yesterday, you could spawn a child instance set to 10:22 AM to extract only the missing data and merge it back.
- **Test and QA environments:** Clone a production-like environment from any point in the last X days (as defined by your restore window) to validate schema changes or run integration tests.
- **Compliance and auditing:** Provision a child instance reflecting the database state at a past date for internal audits, financial reconciliations, or forensic analysis.

What is a child instance?

A child instance is a copy-on-write clone of the original database instance. It can be created from the current point in time or from a historical point-in-time within the *restore window*. A restore window specifies the time range (2–35 days) from which you can perform point-in-time recovery. The restore window is defined when creating a database instance. You can create a child instance at any point within this restore window, without waiting for manual snapshots or full database exports.

After you create a child instance, it is available for query until it is deleted. There is no automatic expiration tied to the parent's restore window. This ensures you always have access to the point-in-time data at which the child instance was created, even if the parent's restore window has expired.

Child instances are isolated from the parent. Any reads or writes you perform on a child instance have no effect on the source instance. Modifications made on a child instance uses copy-on-write so the changes are stored as a "delta". This keeps unmodified data shared and minimizes extra storage.

Create a child instance to time travel and restore data

Creating a child instance is useful in the following scenarios:

- **Accidental data loss recovery:** If a critical table was dropped at 10:23 AM yesterday, you could spawn a child instance set to 10:22 AM to extract only the missing data and merge it back.
- **Test and QA environments:** Clone a production-like environment from any point in the last X days (as defined by your restore window) to validate schema changes or run integration tests.
- **Compliance and auditing:** Provision a child instance reflecting the database state at a past date for internal audits, financial reconciliations, or forensic analysis.

Step by step instructions

Step 1: (Optional) Configure the restore window on a parent database instance

Valid values for the restore window are between 2 to 35 days, inclusive. The default window is 7 days. You can specify the restore window on a database instance when you create the instance. See [Create a database instance](#).

Use the UI or API to modify the restore window of a database instance:

UI **curl**

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to update.
4. Click **Edit** in the upper-right corner.
5. Under **Advanced settings**, set the number of days for the restore window.
6. Click **Save**.

Step 2: Create a child instance

A child instance's roles and permissions are the same as the parent's at the chosen restore point. This might lead to users having privileges that don't match the currently granted privileges on parent instance. [View Databricks identity roles](#) and revoke any outdated privileges on the child instance.

Create a child instance by enabling **Create from parent** during regular database instance creation. See [Create and manage a database instance](#).

UI **curl** **Python SDK** **CLI**

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Click **Create database instance**.
4. Configure your instance:

- **Name:** Enter a database instance name (1–63 characters, letters and hyphens only).
- **Capacity:** Select the compute size (default: 2 CU). See [Instance size](#).
- **Serverless usage policy:** Select a budget policy to attribute serverless usage and billing (optional). See [Serverless usage policies](#).

5. Expand **Advanced Settings**.

6. Enable **Create from parent**.

7. Configure the child instance settings:

- **Parent instance name:** Enter the name of the parent instance you want to create a child database from.
- **Include data from parent up to:** Select the specific historic point in time you want data from.
 - **Current point in time:** Creates a child instance with the most recent data from a database instance.
 - **Specific date and time:** Creates a child instance at a specific point in time within the restore window of a database instance. The child instance contains all the data at that particular point in time.
 - **Specific Log Sequence Number:** Creates a child instance from a particular write-ahead-log (WAL) log sequence number (LSN) in the restore window. PostgreSQL uses WAL to support recovery and assigns an LSN to each WAL record. The child instance contains all the data at that WAL LSN. For more about WALs, see [WAL internals](#). To obtain the current LSN value, see [Backup control functions](#).

8. Click **Create**.

Step 3 (Optional): Restore data

After creating a child instance at a specific point in time, you can restore data from the child instance back to your production instance. This is useful for recovering from accidental data loss or corruption.

To restore data:

1. Connect to the child instance using your preferred PostgreSQL client or connection method. See [Connect and query](#).

2. Write a script to copy the required data from the child instance to your production instance. The script should do the following:
 - Query the specific tables or data you need from the child instance
 - Insert or update the data in your production instance
 - Handle any conflicts or data transformations as needed
3. Test the restore operation on a small subset of data first to ensure correctness.
4. Run the full restore script to copy all required data back to production.
5. Delete the child instance after successfully restoring the data and verifying the production instance is correct. To delete your instance, see [Delete an instance](#).

Limitations

- Only one child instance can be created at a time for a database instance.
- Parent instances cannot be deleted unless the child instance is deleted.
- Child instances are designed for short-term testing or recovery. Heavy modifications on the child or parent can cause the system to reduce data sharing and maintain a full copy for the child. Storage usage and cost can grow to match that of a standalone database instance.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Mar 11, 2026**

Monitor a database instance

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

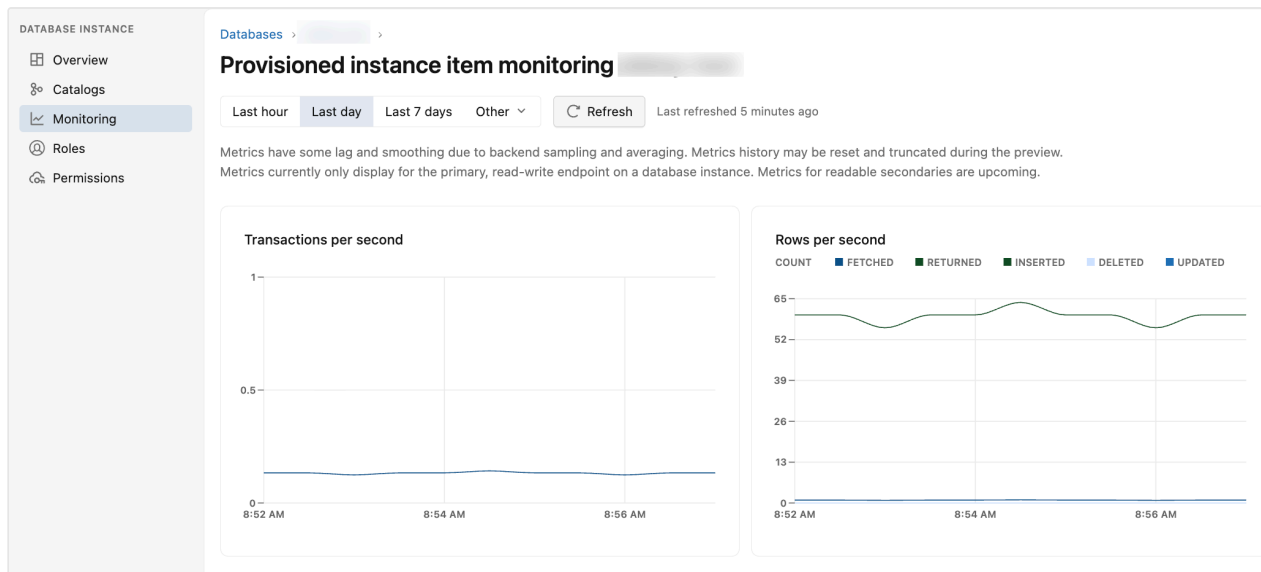
New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page explains how to monitor the performance of your Lakebase database instance using the **Metrics** tab on the instance details page.

Access metrics

To access metrics for your instance:

1. Click **☰ Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to monitor.
4. Select **Monitoring** from the Lakebase App sidebar.



Available metrics

Use the following metrics to analyze performance trends, identify potential bottlenecks, and evaluate whether to optimize application usage or scale your instance:

- **Transactions per second:** Shows committed transaction throughput. Use this to understand workload patterns and identify peak transaction periods. If this value is consistently high, consider optimizing client behavior or increasing the size of the instance.
- **Rows per second:** Displays the number of rows fetched, returned, inserted, updated, and deleted. Rows fetched refers to the number of rows returned to clients. Rows returned refers to the number of rows read by queries. Helps diagnose the type of workload and its impact on the system. If performance is constrained, consider adding indexes or optimizing query patterns.
- **Open connections:** Displays the number of open active connections. Connections consume instance resources. Use this to evaluate whether client-side connection pooling is needed. Refer to limits for the maximum number of allowed connections.
- **Storage utilization:** Indicates current storage usage for the instance. If utilization approaches Lakebase Provisioned limits, remove unnecessary data or indexes. Alternatively, contact support to request a quota increase.

- **CPU utilization (%):** Measures CPU usage for the database instance. High CPU usage might indicate a computation-heavy workload. Consider application-side optimizations or increasing the size of the instance.
- **Page read throughput (%):** Reflects how close the instance is to its page read capacity, typically caused by cache misses. If the **Page read throughput** value is high, reduce the workload or working set, add indexes, cache queries on the client side, or streamline the data.
- **Buffer cache hit rate (%):** Indicates the percentage of reads served from memory. High-performance workloads should see values above 99%. Low rates suggest the workload exceeds cache capacity or could benefit from optimizations.
- **Local SSD cache hit rate (%):** Tracks the percentage of reads served from the SSD cache after a buffer cache miss. A low value may increase page read throughput. Use similar optimizations as with buffer cache, or consider a larger instance.
- **Deadlocks per second:** Measures how often transactions encounter deadlocks. These typically occur when multiple transactions access the same resources in conflicting order. Investigate and refactor workloads to prevent deadlocks.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Mar 11, 2026**

Authentication and permissions

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page provides an overview of authentication methods and permission management for Databricks database instances. Learn how to authenticate users, manage roles, and control access to your database resources.

Authentication methods

Authenticate to your database instance.

Method	Description
Authenticate with Databricks identities	Use OAuth tokens with Databricks users, groups, and service principals for secure access. Best for integrating with Databricks workspace security and automated workflows.
Authenticate with native Postgres	Use traditional Postgres username and password authentication by enabling native Postgres role login for  Ask Genie

Method	Description
roles	your database instance.

Role and permission management

Grant Databricks identities access to your database instance and create corresponding Postgres roles.

NOTE

Databricks identities and Postgres roles are separate systems. There is no automatic sync between them. You need to create Postgres roles for Databricks identities that need PostgreSQL access.

Task	Description
Manage permissions	Learn when and how permissions are checked between Databricks and Postgres systems. Grant Databricks users and identities privileges to a database instance on Databricks.
Manage PostgreSQL roles	Create and manage Postgres roles for Databricks identities. This is essential for enabling PostgreSQL access for users.
Pre-created roles and permissions	Understand pre-created roles, system roles, and their capabilities. Learn about the <code>databricks_superuser</code> role and instance owner privileges.

Like this page

Was this page helpful?

 Yes

 No

 Ask Genie

Last updated on **May 4, 2026**

Authenticate to a database instance

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page describes how to authenticate to a Lakebase database instance. There are two ways to authenticate:

1. Obtain an OAuth token and authenticate using Databricks identities.
2. Use native Postgres roles with passwords.

Authenticate with Databricks identities

When you authenticate as a Databricks identity, you need to generate an OAuth token and use it as a password when connecting to Postgres.

Considerations before you begin

- OAuth tokens expire after one hour, but expiration is enforced only at connection time. Connections remain active even if the token expires. However, any Pu

command that requires authentication fails if the token has expired.

- OAuth tokens used for Postgres authentication are workspace-scoped and should belong to the same workspace that owns the database instance. Cross-workspace token authentication is not supported. To learn more about authentication, see [Authentication for the Databricks CLI](#).
- Token-based authentication requires a plaintext password, so only SSL connections are allowed. Ensure that the client library you use to access Postgres with token-based authentication is configured to establish an SSL connection.

Obtain an OAuth token in a user-to-machine flow

If you are a database owner, admin, or your Databricks identity has a corresponding Postgres role for the database instance, you can obtain an OAuth token from the UI, the Databricks CLI, or one of the Databricks SDKs. You can restrict the scope of the token appropriately using the Databricks CLI.

For other Databricks identity users, see [Authorize user access to Databricks with OAuth](#) for the workspace-level authorization instructions to obtain OAuth tokens.

UI CLI Python SDK Java SDK

When your database instance **Status** is **Available**, use the Databricks UI to obtain an OAuth token:

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Find and click the name of the database instance you want to obtain an OAuth token for.
4. Click the **Connection details** tab.
5. Click **Get OAuth Token**. A flag indicates when the token has been created.
6. Click **Copy OAuthToken** to copy the token to your clipboard.

Obtain an OAuth token in a machine-to-machine flow

To enable secure, automated (machine-to-machine) access to the database instance, you must obtain an OAuth token using a Databricks service principal. This process involves configuring the service principal, generating credentials, and minting OAuth tokens for authentication.

1. Configure a service principal with indefinitely lived credentials. For instructions, see [Authorize service principal access to Databricks with OAuth](#).
2. Mint new OAuth tokens as the service principal.

CLI Python SDK Java SDK

When your database instance **Status** is **Available**, use the Databricks CLI v0.256.0 and later to obtain an OAuth token:

- i. Use the following command to fetch a token.

Bash

```
databricks database generate-database-credential \  
  --request-id $(uuidgen) \  
  --json '{"instance_names": ["db-instance-name"]}'
```

- ii. This generates a response in the following format. Copy the `token` from the response.

JSON

```
{  
  "expiration_time": "2025-08-24T14:15:22Z",  
  "token": "<string>"  
}
```

Rotate OAuth tokens before hourly expiration:

- Check the expiration time of the OAuth token on each use and refresh when needed.
- Alternatively, set up a background thread to refresh the current OAuth token periodically.

Troubleshoot token identity

If your token's identity does not match the security label, you might receive a error message like the following.

```
A valid oauth token was supplied but the token's identity "<USER>" did not match the security label configured for role "<SERVICE PRINCIPAL>". Please ensure that the token is generated for the correct databricks identity.
```

Run the following command to check which Databricks identity is returned:

CLI Python SDK Java SDK

Bash

```
databricks current-user me
```

If the expected identity is not returned, verify that the correct credentials were used when initializing the workspace client.

Authenticate as Databricks group

Groups and group memberships are not synced from Databricks into Postgres and neither are Unity Catalog permissions. However, after a Databricks group is added into Postgres, any Databricks user in the group can log in as the group using the user's password. This allows you to manage permissions at the group level in Postgres. Any

direct or indirect member (user or service principal) of the Databricks group identity can authenticate to Postgres and log in as the Databricks group Postgres role.

When authenticating as a group identity using a user or service principal token, group membership is validated only at the time of authentication. Any open connection with a group member token remains open if the member is removed from the group after authentication. Any new connection request from a removed group member is rejected during authentication.

Bash

```
export PGPASSWORD='<0Auth token of a group member>'
export GROUPROLENAME=<pg-case-sensitive-group-role-name>

psql -h $HOSTNAME -p 5432 -d databricks_postgres -U $GROUPROLENAME
```

Only groups assigned to the Databricks workspace of the database instance are supported for group-based Postgres login. To learn how to assign a group to a workspace, see [Assign a group to a workspace](#).

Authenticate with Postgres roles and passwords

If you have clients that do not support credential rotation after one hour, you can create native Postgres roles with passwords:

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to update.
4. Click **Edit** in the upper-right.
5. Turn on **Enable Postgres Native Role Login**.
6. Click **Save**.

7. Log into Postgres, or use the SQL Editor, to create a role with a password.

PostgreSQL

```
CREATE ROLE new_role LOGIN PASSWORD 'your strong password';
```

8. Grant additional Postgres permissions to the new role. See [grant privileges to Postgres roles using PostgreSQL](#).

Next steps

After obtaining a credential (OAuth token or password), you can connect to your database instance:

- [Access a database instance from SQL clients](#)
- [Use a notebook to access a database instance](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Mar 11, 2026**

Manage permissions

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page describes when and how to grant Databricks users and identities permissions to a database instance.

To allow other users to use PostgreSQL to access the database instance, the `databricks_superuser` must create corresponding Postgres roles for them. For details on how to create Postgres roles, see [Manage PostgreSQL roles](#).

When and how permissions are checked

When you use Postgres syntax or connect through a PostgreSQL interface, Lakebase enforces PostgreSQL-specific access controls by using the following:

- Postgres roles
- Role memberships
- Postgres-granted permissions

In all other scenarios, Lakebase enforces Databricks-specific access control

- Databricks identities (users, groups and service principals)
- Databricks group memberships
- Workspace access control lists (ACLs)
- Unity Catalog privileges

NOTE

There is no automatic sync between Databricks identities and memberships, and Postgres roles and memberships.

Use case / Permission or identity	Manage database instances	Create or delete synced tables	Manage synced table pipeline	Query Postgres tables from a SQL warehouse	Query online features in feature and model serving	Query Postgres tables from a SQL warehouse
Databricks identities	✓	✓	✓	✓	✓	Rec con Pos
Databricks group memberships	✓	✓	✓	✓	✓	On on log gro
Instance ACLs	✓	✓				
Pipeline ACLs		Need to be a pipeline owner when reusing	✓			

Use case / Permission or identity	Manage database instances	Create or delete synced tables	Manage synced table pipeline	Query Postgres tables from a SQL warehouse	Query online features in feature and model serving	Query Postgres tables from a SQL warehouse
		an existing pipeline or deleting a synced table (which edits the pipeline)				
UC permissions		✓		✓	✓	
Postgres roles						✓
Postgres role memberships						✓
Postgres permissions						✓

Database instance permissions

A user must have specific permissions on the database instance to manage the instance and perform table operations from the Databricks UI, API, or SDK. Workspace admins and the instance creator can assign additional permissions to any desired users, groups, or service principals in the **Database instances** overview page.

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select your database instance.
4. Select the **Permissions** page in the Lakebase App sidebar.
5. Click **Grant permission** in the upper-right.
6. From the **Principal** drop-down menu, select a user, group, or service principal to grant permissions to.
7. From the **Permission** drop-down menu, select the permission level you want to grant. See [Database instance ACLs](#).
8. Click **Grant**.

Postgres permissions

To read from or write to a table in Postgres, users require the following permissions in Postgres:

- `CONNECT` on the database
- `USAGE` on the schema (or `CREATE` to create new tables)
- `SELECT`, `INSERT`, `UPDATE`, or `DELETE` on the table

For the full list of Postgres privileges, see [the PostgreSQL documentation](#).

Check user permissions on a specific table in Postgres

Run the following SQL statement to create a function named `pg_temp.check_permissions` that checks a user's permissions, including inherited permissions.


```

CREATE OR REPLACE FUNCTION pg_temp.check_permissions(p_user TEXT,
p_table_name TEXT)
RETURNS TABLE(database_connect BOOLEAN, schema_usage BOOLEAN,
table_select BOOLEAN,
                table_insert BOOLEAN, table_update BOOLEAN, table_delete
BOOLEAN) AS $$
WITH p AS (SELECT string_to_array(p_table_name, '.') AS a),
c AS (
    SELECT
        COALESCE(a[1], current_schema()) AS sc,
        COALESCE(a[2], a[1]) AS tb
    FROM p
)
SELECT
    has_database_privilege(p_user, current_database(), 'CONNECT'),
    has_schema_privilege(p_user, sc, 'USAGE'),
    has_table_privilege(p_user, sc || '.' || tb, 'SELECT'),
    has_table_privilege(p_user, sc || '.' || tb, 'INSERT'),
    has_table_privilege(p_user, sc || '.' || tb, 'UPDATE'),
    has_table_privilege(p_user, sc || '.' || tb, 'DELETE')
FROM c
$$ LANGUAGE sql;

```

To use the function, use the following query:

PostgreSQL

```

SELECT * FROM pg_temp.check_permissions('<your_user>', '<your_table>');

-- Example:
SELECT * FROM pg_temp.check_permissions('joe@acme.org', 'public.table1');

```

View all permissions for database, schema, and table in Postgres

The following SQL statement creates a function named `pg_temp.make_owner_acl` that returns a detailed view of all granted permissions, excluding inherited permissions, for all users:

PostgreSQL

```

CREATE OR REPLACE FUNCTION pg_temp.make_owner_acl(owner_oid OID)
RETURNS TABLE(grantor OID, grantee OID, privilege_type TEXT, is_grantable
BOOLEAN) AS $$
SELECT owner_oid, owner_oid, 'OWNER'::TEXT, True
$$ LANGUAGE SQL;

CREATE OR REPLACE FUNCTION pg_temp.get_all_permissions(TEXT, TEXT, TEXT)
RETURNS TABLE(type TEXT, name TEXT, role TEXT, acl TEXT, can_grant TEXT)
AS $$
SELECT type, name,
CASE WHEN grantee = 0 THEN 'PUBLIC' ELSE pg_get_userbyid(grantee) END
AS role, privilege_type,
CASE WHEN privilege_type = 'OWNER' THEN 'YES, ALL' WHEN is_grantable
THEN 'YES' ELSE 'NO' END AS can_grant
FROM (
SELECT 'DATABASE' type, datname as name,
(pg_temp.make_owner_acl(datdba)).*
FROM pg_database
WHERE datname = $1
UNION ALL
SELECT 'DATABASE' type, datname as name, (aclexplode(datacl)).*
FROM pg_database
WHERE datname = $1
UNION ALL
SELECT 'SCHEMA' type, nspname as name,
(pg_temp.make_owner_acl(nspowner)).*
FROM pg_namespace
WHERE nspname = $2
UNION ALL
SELECT 'SCHEMA' type, nspname as name, (aclexplode(nspacl)).*
FROM pg_namespace
WHERE nspname = $2
UNION ALL
SELECT 'TABLE' type, relname as name,
(pg_temp.make_owner_acl(reloverner)).*
FROM pg_class
WHERE relname = $3
UNION ALL
SELECT 'TABLE' type, relname as name, (aclexplode(relacl)).*
FROM pg_class
WHERE relname = $3
)
$$ LANGUAGE SQL
;

```

To use the function, use the following query:

PostgreSQL

```
SELECT * FROM pg_temp.get_all_permissions('<your_database>',
'<your_schema>', '<your_table>');

-- Example:
SELECT * FROM pg_temp.get_all_permissions('databricks_postgres',
'public', 'table1');
```

Check role inheritance hierarchy in Postgres

In Postgres, a role can be a member of another role, and the membership specifies whether the permissions are inherited from the parent role. To see all the roles that a certain role is part of, use the following SQL statement to create the SQL function

`pg_temp.get_inherited_roles`:

PostgreSQL

```
CREATE OR REPLACE FUNCTION pg_temp.get_inherited_roles(
    role_name TEXT
)
RETURNS TABLE(inherited_roles TEXT, member_via TEXT, inherits_permissions
TEXT) AS $$
    WITH RECURSIVE role_tree AS (
        SELECT
            m.roleid,
            pg_get_userbyid(m.roleid) rolname,
            'DIRECT' COLLATE "C" as member_via,
            m.inherit_option as inherits_permissions
        FROM pg_auth_members m
        WHERE m.member = (SELECT oid FROM pg_roles WHERE rolname = $1)
        UNION ALL
        SELECT
            m.roleid,
            pg_get_userbyid(m.roleid) rolname,
            rt.rolname::text as member_via,
            (rt.inherits_permissions AND m.inherit_option) as
inherits_permissions
        FROM pg_auth_members m
        JOIN role_tree rt ON m.member = rt.roleid
    )
    SELECT
        rolname AS inherited_roles,
        member_via,
```

```

CASE WHEN inherits_permissions THEN 'YES' ELSE 'NO' END as
inherits_permissions
FROM role_tree
GROUP BY inherited_roles, member_via, inherits_permissions
ORDER BY inherits_permissions DESC
$$ LANGUAGE sql
;

```

To use the function, substitute a Databricks username for `<your_user>`:

PostgreSQL

```
SELECT * FROM pg_temp.get_inherited_roles('<your_role>');
```

Find role administrators in Postgres

To see the admin of a role, use the following SQL query and substitute the role name for `<target_role>`:

PostgreSQL

```

SELECT pg_get_userbyid(m.member) admin
FROM pg_auth_members m
WHERE m.roleid = (SELECT oid FROM pg_roles WHERE rolname =
'<target_role>')
AND m.admin_option = true;

```

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Mar 11, 2026**

Manage PostgreSQL roles

📘 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

A Postgres role for the Lakebase database instance owner's Databricks identity is created automatically.

Initially, only the owner of the instance can log in and access the instance through Postgres. To allow other Databricks identities to log in to the database instance, the Databricks identity must have a corresponding Postgres role.

This page describes how to add and manage additional Databricks identity-based roles in PostgreSQL.

Create Postgres roles and grant privileges for Databricks identities

Create Postgres roles using the Databricks UI or PostgreSQL queries.

📌 NOTE

 Ask Genie

Role management actions are governed by the permissions granted on the database instance. Ensure you have the appropriate level of access before attempting to manage roles.

UI PostgreSQL

Users with **CAN USE** permission on the database instance can view the existing Postgres roles associated with Databricks identities or add a role for their own identity to the instance.

Users with **CAN MANAGE** can additionally create roles for other Databricks identities, including with additional privileges, and drop roles for any Databricks identity.

You can assign additional permissions to any desired users, groups, or service principals in the **Database instances** overview page.

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select your database instance.
4. Select the **Roles** page in the Lakebase App sidebar.
5. Click **Add role** in the top right corner.
6. From the **Principal** drop-down menu, select a user, group, or service principal that you want to grant database access. You must select a Databricks identity that doesn't already have a Postgres role in the instance.
7. (Optional) Select **System Roles** to grant:
 - **databricks_superuser**: Grants read and write access to all data. This is an administrative role with broad privileges that inherits from `pg_read_all_data`, `pg_write_all_data`, and `pg_monitor`. See [Pre-created roles and permissions](#).
8. (Optional) Select **System Attributes** to grant specific capabilities:
 - **CREATEDB**: Grants permission to create new databases.
 - **CREATEROLE**: Grants permission to create new roles.
 - **BYPASSRLS**: Grants permission to bypass row-level security (RLS) policies.
9. Click **Add**.

View Databricks identity roles

UI PostgreSQL

You can see which users, groups, and service principals has a corresponding Postgres role in the **Database instances** overview page.

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select your database instance.
4. Select the **Roles** page in the Lakebase App sidebar.

Drop a Databricks identity-based Postgres role

UI PostgreSQL

Dropping a role cannot be undone. You can recreate a role, but any object ownership reassignment is non-reversible without dropping the new role that owns reassigned objects.

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select your database instance.
4. Select the **Roles** page in the Lakebase App sidebar.
5. For the role you want to drop, click  on the right side of the row.
6. Click **Drop**.
7. (Optional) If the role owns objects, turn on **Reassign owned objects** to enable dropping the role. This option reassigns owned objects to another role before dropping. Some objects (such as grants to the role being dropped) aren't reassignable and are dropped after reassignment of all reassignable objects.
8. Click **Confirm**.

Is this page

as this page helpful?

☐ Yes	☐ No
☐ Send feedback	

Last updated on **Mar 11, 2026**

Pre-created roles and permissions

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page explains the Postgres roles that you can use to govern access to a Databricks Lakebase database instance, including their privileges, purpose, and configuration.

Pre-created roles

After a database instance is created, Databricks automatically creates a Postgres role for the user who created the instance.

Role	Description	Inherited privileges
<instance_owner_role>	The Databricks identity of the instance creator (for example, <code>myuser@databricks.com</code>). This role can log in and administer the instance.	Member of <code>databricks_superuser</code>
<code>databricks_superuser</code>	An internal administrative role. Used to configure and manage access across the instance. This role is granted broad privileges and should not be used in automated applications.	Inherits from <code>pg_read_all_data</code> , <code>pg_write_all_data</code> , and <code>pg_monitor</code> .

Role capabilities

Role	LOGIN	CREATEDB	CREATEROLE	BYPASSRLS
<code>databricks_superuser</code>	NOLOGIN	—	—	—
<instance_owner_role>	✓	✓	✓	✓

System roles created by Databricks

In addition to the `databricks_superuser` and admin roles, Databricks creates system roles required for internal services. These roles are assigned the minimum privileges required for functionality. Modifying them can impact instance behavior.

Role	Purpose
<code>databricks_control_plane</code>	Used by internal Databricks components for management operations
<code>databricks_monitor</code>	Used by internal metrics collection services
<code>databricks_writer_<dbid></code>	Per-database role used to create and manage synced tables
<code>databricks_reader_<dbid></code>	Per-database role used to read tables registered in Unity Catalog
<code>databricks_gateway</code>	Used for internal connections for managed data serving services

To learn how roles, privileges, and role memberships work in Postgres, use the following resources in the Postgres documentation:

- [Database Roles](#)
- [Privileges](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Mar 11, 2026**

Work with Unity Catalog

Integrate your Lakebase Provisioned database with Unity Catalog to enable data governance, federated queries, and automated data synchronization between your Lakehouse and PostgreSQL databases.

How to integrate with Unity Catalog

Learn how to connect Lakebase with Unity Catalog for enhanced data management and governance.

Feature	Description
Register your database in Unity Catalog	Register your Lakebase database as a read-only Unity Catalog catalog to enable federated queries and data governance.
Serve lakehouse data with synced tables	Create synced tables to serve Unity Catalog data through your Lakebase database instance for operational applications.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

Last updated on **May 11, 2026**

Register your database in Unity Catalog

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page explains how to register your Lakebase database as a read-only Unity Catalog catalog. This allows you to use Unity Catalog privileges to manage data access and leverage integrations like managed data syncing.

Ownership and permissions

- If the database does *not* exist, it can be created along with the catalog. In this case, ownership of the objects will follow [these guidelines](#).
- The catalog creator must additionally have `CREATE CATALOG` privileges on the Unity Catalog metastore.

Create a catalog

To register a database with Unity Catalog, use the Databricks UI, API call, Python SDK, or CLI.

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select your database instance.
4. Select the **Catalogs** page in the Lakebase App sidebar.
5. Click **Add catalog** in the upper-right.
6. In the **Add catalog** dialog:
 - **Catalog name:** Enter the desired Unity Catalog catalog name.
 - **Postgres database:** Select an existing Postgres database from the drop-down menu (for example, `databricks_postgres`), or enter a new database name to create it along with the catalog.
7. Click **Create**.
8. After creation, click on the catalog in the **Catalogs** list to see the Catalog Explorer view.

Limitations

- Database names must only contain alphanumeric or underscore characters. Note database names cannot include hyphens.
- The catalog created in Unity Catalog based on the database is read-only.
- Database instances are scoped to a single workspace and don't support cross-workspace access. Users are able to see these tables in Catalog Explorer if they have the required Unity Catalog permissions from other workspaces attached to the same metastore, but they cannot access the table contents.
- Catalog names must follow the [existing constraints](#) for Unity Catalog securable object names.

Explore Postgres objects in Unity Catalog via a Database Catalog

To view the catalog, ensure you have a running serverless SQL warehouse attached.

1. Click **Catalog** in the workspace sidebar.
2. From the **Catalog** browser, click the cluster and select a running serverless SQL warehouse as your compute resource.
3. Click **Start and close**.

As you open the catalog, schemas, and tables, syncs should be triggered automatically. However, the UI may cache data to reduce the number of Postgres requests, so new objects may not show up immediately.

To trigger a full refresh, click .

Delete a database catalog in Unity Catalog

You must be the owner of a database catalog or a metastore admin to delete it.

Databricks recommends that you first delete all synced tables from the database catalog. Each source table can only be used to create 20 synced tables, and it can take up to three days for synced tables to be cleaned up in Unity Catalog after a database catalog is deleted. Synced tables pending deletion in Unity Catalog count toward the 20 synced tables per source table limit.

Use the Databricks UI, Python SDK, CLI, or API to delete a database catalog.

UI Python SDK CLI curl

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select your database instance.
4. Select the **Catalogs** page in the Lakebase App sidebar.
5. For the catalog you want to delete, click  on the right side of the row.
6. Click **Delete**.

After the catalog is deleted, you can rename or drop the Postgres database.

Is this page

Was this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Apr 15, 2026**

Serve lakehouse data with synced tables (Lakebase Provisioned)

! INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

Synced tables let you serve lakehouse data through Lakebase Provisioned Postgres instances. Unity Catalog tables sync into Postgres so applications can query lakehouse data directly with low latency. This process is commonly known as reverse ETL. The lakehouse is optimized for analytics and enrichment, while Lakebase is designed for operational workloads that require fast lookup-style queries and transactional consistency.

What are synced tables?

Synced tables let you serve analytics-grade data from Unity Catalog through Lakebase Provisioned Postgres, making it available to applications that need low-latency queries and full ACID transactions. They bridge the gap between analytical storage and operational systems by keeping your data ready to serve in real-time applications.

Sync pipelines use managed Lakeflow Spark Declarative Pipelines to continuously update both the Unity Catalog synced table and the Postgres table with changes from the source table. After creation, synced tables can be queried directly using Postgres tools.

The key characteristics of synced tables are as follows:

- The table is treated as read-only in Postgres to maintain data integrity with the source
- Automatically synchronized using managed Lakeflow Spark Declarative Pipelines
- Queryable through standard PostgreSQL interfaces
- Managed through Unity Catalog for governance and lifecycle management

WARNING

While it's possible to modify a synced table directly in Postgres, Databricks strictly recommends running only read queries to protect data integrity with the source. For supported operations on synced tables, see [Supported operations](#).

Before you begin

- You have a Unity Catalog table in any catalog.
- You have `CAN USE` permissions on the database instance.

Create a synced table

UI Python SDK CLI curl

To sync a Unity Catalog table into Postgres, do the following:

1. Click **Catalog** in the workspace sidebar.
2. Find and select the Unity Catalog table you want to create a synced table on.
3. Click **Create** > **Synced table**.

 Ask Genie

4. Select your catalog, schema, and enter a table name for the new synced table.
 - Synced tables can also be created in [Standard catalogs](#), with some additional configuration. Select your Standard catalog, a schema, and enter a table name for the newly created synced table.
5. Select a database instance and enter the name of the Postgres database in which to create the synced table. The Postgres database field defaults to the currently selected target catalog. If a Postgres database does not exist under this name, Databricks creates a new one.
6. Select a **Primary Key**. A primary key is *required* as it enables efficient access to rows for reads, updates, and deletes.

❗ IMPORTANT

Columns in the primary key are not nullable in the synced table. Therefore, rows with nulls in primary key columns are *excluded from the sync*.

7. If two rows have the same primary key in the source table, select a **Timeseries Key** to configure deduplication. When a **Timeseries Key** is specified, the synced tables contains only the rows with the latest timeseries key value for each primary key.
8. Select the sync mode from **Snapshot**, **Triggered**, and **Continuous**. For more information about each sync mode, see [Sync modes explained](#).
9. Choose if you want to create this synced table from a new or existing pipeline.
 - If creating a new pipeline and using a managed catalog, choose the storage location for the staging table. If using a standard catalog, the staging table is automatically stored in the catalog.
 - If using an existing pipeline, check that the new sync mode matches the pipeline mode.
10. (Optional) Select a **Serverless usage policy**. To create a serverless usage policy, see [Attribute usage with serverless usage policies](#). This allows you to attribute billing usage to specific serverless usage policies.

- For synced tables, the billable entity is the underlying Lakeflow Spark Declarative Pipelines pipeline. To modify the budget policy, modify the underlying pipeline object. See [Configure a serverless pipeline](#).

11. After **Synced table status** is **Online**, log in to your database instance and query the newly created table. Query your table using the [SQL editor](#), [external tools](#), or [notebooks](#).

Sync modes explained

A synced table can be created with one of the following sync modes, which determine how data is synchronized from the source to the synced table in Postgres:

Sync mode	Description	When to use	Performance
Snapshot	The pipeline runs once to take a snapshot of the source table and copy it to the synced table. Subsequent pipeline runs copy the whole source data to the destination and replace it in place atomically. The pipeline can be triggered manually, through an API or on a schedule.	Source changes >10% of rows per cycle, or the source does not support Change Data Feed (views, Iceberg tables).	10x more efficient than incremental modes for high-churn tables.
Triggered	The pipeline runs once to take a snapshot of the source table and copy it to the synced table. Unlike the Snapshot sync mode, when the synced table is refreshed, only changes since the last	Source rows change incrementally on a known cadence. Inserts, updates, and deletes are all	Good cost/lag tradeoff for on-demand or scheduled syncs. Running more frequently than every 5 minutes may cost more

Sync mode	Description	When to use	Performance
	pipeline execution are retrieved and applied to the synced table. The incremental refresh can be triggered manually, through an API or on a schedule.	propagated on each refresh.	than Continuous mode.
Continuous	The pipeline runs once to take a snapshot of the source table and copy it to the synced table then the pipeline runs continuously. Subsequent changes to the source table are incrementally applied to the synced table in real time. No manual refresh is necessary.	Changes must appear in Lakebase in near real time.	Lowest lag, highest cost.

📌 NOTE

To support **Triggered** or **Continuous** sync mode, the source table must have [Change data feed](#) enabled. Certain sources (like Views) do not support change data feed so they can only be synced in Snapshot mode.

Schedule or trigger subsequent syncs

The initial snapshot runs automatically on creation. For **Snapshot** and **Triggered** modes, subsequent syncs must be triggered explicitly. **Continuous** mode is self-managing.

Database Table Sync pipeline task

The **Database Table Sync pipeline** task in Lakeflow Jobs runs a synced table's pipeline as a workflow step. Configure the job with a table update trigger or a schedule.

Trigger on source table updates

Fires the job when the source Unity Catalog table is updated. With **Triggered** mode, only new changes are applied incrementally, giving near-real-time freshness without the always-on cost of Continuous mode.

1. In the sidebar, click **Workflows**.
2. Click **Create job** or open an existing job.
3. On the **Tasks** tab, click **+ Add another task type**.
4. Under **Ingestion and Transformation**, select **Database Table Sync pipeline**.
5. In the **Pipeline** field, select the pipeline associated with your synced table.
6. Under **Schedules & Triggers**, click **Add trigger**.
7. Select **Table update** as the trigger type.
8. Under **Tables**, select the source Unity Catalog table to monitor.
9. Click **Save**.

Trigger on a schedule

Runs the sync at a fixed cadence. Well-suited for **Snapshot** mode, where a nightly or weekly full refresh is typically the most efficient pattern.

1. Follow steps 1–5 above to add a **Database Table Sync pipeline** task to a job.
2. Under **Schedules & Triggers**, click **Add trigger**.
3. Select **Scheduled** as the trigger type.
4. Set your cron schedule and timezone, then click **Save**.

Using the SDK

Trigger a sync run programmatically, for example at the end of an upstream notebook or pipeline:



Python

```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

# Get the pipeline ID from the synced table
table =
w.database.get_synced_database_table(name="catalog.schema.synced_table")
pipeline_id = table.data_synchronization_status.pipeline_id

# Trigger a sync run
w.pipelines.start_update(pipeline_id=pipeline_id)
```

Supported operations

Databricks recommends performing only the following operations in Postgres for synced tables to prevent accidental overwrites or data inconsistencies:

- Read-only queries
- Creating indexes
- Dropping the table (to free up space after removing the synced table from Unity Catalog)

while it's possible to modify synced tables in Postgres in other ways, it interferes with the synchronization pipeline.

Delete a synced table

To delete a synced table, you must delete it from Unity Catalog and then drop the table in the database instance. Deleting the synced table from Unity Catalog deregisters the table and stops any data refreshes. However, the table remains in the underlying Postgres database. To free up space in your database instance, connect to the instance and use the `DROP TABLE` command.

1. Click **Catalog** in the workspace sidebar.
2. Find and select the synced table you want to delete.
3. Click **> Delete**.
4. Connect to the instance with `psql`, the SQL editor, or from a notebook.
5. Drop the table using PostgreSQL.

PostgreSQL

```
DROP TABLE synced_table_database.synced_table_schema.synced_table
```

Ownership and permissions

If you create a new Postgres database, schema, or table, Postgres ownership is set as follows:

- Ownership is assigned to the user creating the database, schema, or table, if their Databricks login exists as a role in Postgres. To add a Databricks identity role in Postgres, see [Manage PostgreSQL roles](#).
- Otherwise, ownership is assigned to the owner of the parent object in Postgres (typically the `databricks_superuser`).

Manage synced table access

After a synced table is created, the `databricks_superuser` can `READ` a synced table from Postgres. The `databricks_superuser` has `pg_read_all_data`, which lets this role read from all tables. It also has the `pg_write_all_data` privilege, which lets this role write to all tables. This means that a `databricks_superuser` can also write to a synced table in Postgres. Lakebase supports this writing behavior in case you need to make urgent changes in your target table. However, Databricks recommends making fixes in your source table instead.

- The `databricks_superuser` can also grant these privileges to other users:

PostgreSQL

```
GRANT USAGE ON SCHEMA synced_table_schema TO user;
```

 Ask Genie

PostgreSQL

```
GRANT SELECT ON synced_table_name TO user;
```

- The `databricks_superuser` can revoke these privileges:

PostgreSQL

```
REVOKE USAGE ON SCHEMA synced_table_schema FROM user;
```

PostgreSQL

```
REVOKE {SELECT | INSERT | UPDATE | DELETE} ON synced_table_name FROM user;
```

Manage synced table operations

The `databricks_superuser` can manage which users are authorized to perform specific operations on a synced table. The supported operations for synced tables are:

- `CREATE INDEX`
- `ALTER INDEX`
- `DROP INDEX`
- `DROP TABLE`

All other DDL operations are denied for synced tables.

To grant these privileges to additional users, the `databricks_superuser` must first create an extension on `databricks_auth`:

PostgreSQL

```
CREATE EXTENSION IF NOT EXISTS databricks_auth;
```

 Ask Genie

Then the `databricks_superuser` can add a user to manage a synced table:

PostgreSQL

```
SELECT  
databricks_synced_table_add_manager('synced_table_schema"."synced_table" '::  
'[user]');
```

The `databricks_superuser` can remove a user from managing a synced table:

PostgreSQL

```
SELECT databricks_synced_table_remove_manager('[table]', '[user]');
```

The `databricks_superuser` can view all managers:

PostgreSQL

```
SELECT * FROM databricks_synced_table_managers;
```

Data type mapping

This type mapping table defines how each [data type](#) in the source Unity Catalog table is mapped to the destination sync table in Postgres:

Source column type	Postgres column type
BIGINT	BIGINT
BINARY	BYTEA
BOOLEAN	BOOLEAN
DATE	DATE
DECIMAL(p,s)	NUMERIC
 Ask Genie	

Source column type	Postgres column type
DOUBLE	DOUBLE PRECISION
FLOAT	REAL
INT	INTEGER
INTERVAL intervalQualifier	INTERVAL
SMALLINT	SMALLINT
STRING	TEXT
TIMESTAMP	TIMESTAMP WITH TIME ZONE
TIMESTAMP_NTZ	TIMESTAMP WITHOUT TIME ZONE
TINYINT	SMALLINT
GEOGRAPHY(srid)	NOT SUPPORTED
GEOMETRY(srid)	NOT SUPPORTED
ARRAY < elementType >	JSONB
MAP < keyType,valueType >	JSONB
STRUCT < [fieldName : fieldType [NOT NULL] [COMMENT str][, ...]] >	JSONB
VARIANT	JSONB
OBJECT	NOT SUPPORTED

NOTE

- The mapping for ARRAY, MAP, and STRUCT types was changed in May 2025. Sync tables created prior to that continue to map those types to JSON.
- The mapping for TIMESTAMP was changed in August 2025. Sync tables created prior to that continue to map it to TIMESTAMP WITHOUT TIME ZONE.

Handle invalid characters

Certain characters, such as the null byte (0x00), are allowed in `STRING`, `ARRAY`, `MAP`, or `STRUCT` columns in Delta tables but are not supported in Postgres `TEXT` or `JSONB` columns. As a result, syncing such data from Delta to Postgres can lead to insertion failures with errors:

```
org.postgresql.util.PSQLException: ERROR: invalid byte sequence for encoding "UTF8": 0x00
```

```
org.postgresql.util.PSQLException: ERROR: unsupported Unicode escape sequence DETAIL: \u0000 cannot be converted to text.
```

- The first error occurs when a null byte appears in a top-level string column, which maps directly to Postgres `TEXT`.
- The second error occurs when a null byte appears in a string nested inside a complex type (`STRUCT`, `ARRAY`, or `MAP`), which Databricks serializes as `JSONB`. During serialization, all strings are cast to Postgres `TEXT`, where `\u0000` is disallowed.

How to resolve:

You can address this issue in one of the following ways:

- Sanitize string fields

Remove or replace unsupported characters from all string fields, including those inside complex types, before syncing to Postgres.

To remove null bytes from a top-level `STRING` column, use the `REPLACE` function;

 Ask Genie

SQL

```
SELECT REPLACE(column_name, CAST(CHAR(0) AS STRING), '') AS  
cleaned_column FROM your_table;
```

- Convert to binary (for `STRING` columns only)

If preserving the raw byte content is necessary, convert the affected `STRING` columns to `BINARY`.

Limitations and considerations

Supported source tables

Depending on the sync mode of a synced table, different types of source tables are supported:

- For Snapshot mode, the source table must support `SELECT *`. Examples include Delta tables, Iceberg tables, views, materialized views, and other similar types.
- For Triggered or Continuous sync modes, the source table must *also* have [change data feed](#) enabled.

Naming and identifier limitations

- **Allowed characters:** Postgres database, schema, and table names for synced tables may only contain alphanumeric characters and underscores (`[A-Za-z0-9_]+`). Hyphens (`-`) and other special characters are not supported.
- **Column and table identifiers:** Avoid using uppercase letters or special characters in column or table names in the source Unity Catalog table. If kept, you need to quote these identifiers when referencing them in Postgres.

Performance and synchronization

- **Sync speed:** Synchronizing data into synced tables may be slower than writing data directly to the database instance with a native PostgreSQL client due to  Ask Genie

additional processing. Continuous sync mode refreshes data from the Unity Catalog managed table to the synced table at a minimum interval of 15 seconds.

- **Connection usage:** Each table synchronization can use up to 16 connections to the database instance, which count toward the instance's connection limit.
- **API idempotency:** Synced table APIs are idempotent and may need to be retried if errors occur to ensure timely operations.
- **Schema changes:** For synced tables in `Triggered` or `Continuous` mode, only additive schema changes (e.g., adding a new column) from the Unity Catalog tables are reflected on the synced table.
- **Duplicate keys:** If two rows have the same primary key in the source table, the sync pipeline fails unless you configure deduplication using a timeseries key. However, using a timeseries key has a performance penalty.
- **Update rate:** For Lakebase Provisioned, the sync pipeline supports Continuous and Triggered writes at approximately 1200 rows per second per Capacity Unit (CU) and Snapshot writes at up to 15,000 rows per second per CU.

Capacity and limits

- **Table limits:**
 - Limit of 20 synced tables per source table.
 - Each table synchronization may use up to 16 database connections.
- **Size limits and full refresh:**
 - If you full-refresh a synced table, the old version in Postgres is not deleted until the new table is synced. Both versions temporarily count toward the logical database size limit during the refresh.
 - Individual synced tables do not have a limit, but the total logical data size limit across all tables in the instance is 2 TB. However, if you require refreshes instead of full table recreation, Databricks recommends not exceeding 1 TB.
 - If the uncompressed, row-format size of the Unity Catalog table exceeds the database instance size limit (2 TB), synchronization fails. You must drop the synced table in Postgres before writing further to the instance.

Catalog integration

- **Catalog duplication:** Creating a synced table in a standard catalog targeting a Postgres database that is also registered as a separate database catalog causes

the synced table to appear in Unity Catalog under both the standard and the database catalogs.

Is this page

as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Mar 11, 2026**

Connect and query

❗ INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page outlines the different ways to work with your Lakebase database instance and recommends how to optimize PostgreSQL queries.

Ways to access your Database

Here are the following methods to access your database instance to run PostgreSQL queries and manage identities. As a managed Postgres service, there are some limitations in functionality. See [PostgreSQL compatibility](#).

Before connecting to your database using an external editor or Databricks notebooks, make sure you can [authenticate to the database instance](#).

❗ NOTE

To connect Classic compute to PostgreSQL, open TCP port 5432 in your network security group to your workspace IP ACL address or range.

Method	Description
Authenticate	Authenticate to a Lakebase database using OAuth tokens for Databricks identities (including machine-to-machine access) or native Postgres roles.
SQL editor	Connect to and query your database from the Databricks SQL editor.
Databricks notebooks	Connect to and query from your database from Databricks notebooks.
SQL clients	Connect to and query from your database from external tools such as <code>psql</code> , DBeaver, and pgAdmin.

If you query through a Databricks SQL endpoint, the query is federated to a database instance.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Mar 11, 2026**

Access a database instance from the SQL editor

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page describes how to access a Lakebase database instance from the SQL editor to run PostgreSQL commands and queries.

Before you begin

Verify that you meet the following requirements before connecting:

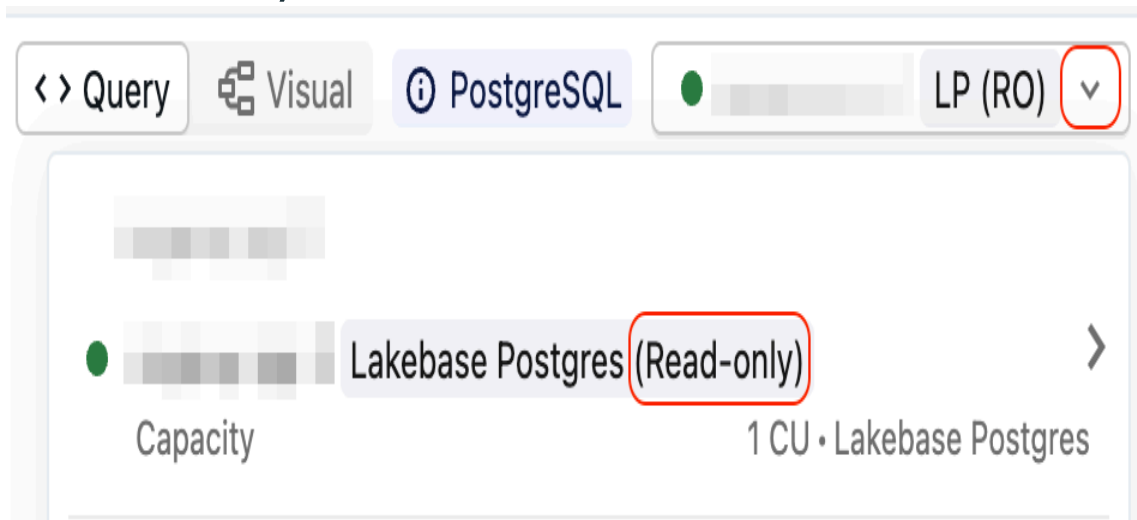
- You have a corresponding Postgres role to log in to the database instance. See [Manage PostgreSQL roles](#).
- Your Postgres role is granted the necessary permissions to access the database, schema, or table.

The database instance creator already has the necessary permissions to access the database instance from the SQL editor. The instance creator can also add additional users to the instance and can grant Postgres permissions to the user.

Create a new query

Create a new SQL query in the SQL editor.

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to run PostgreSQL queries on.
4. Click **New Query** in the upper-right.
5. To run a read-only query on [readable secondaries](#):
 - i. Click the compute resources drop-down menu and select your compute that has the **(Read-only)** label.



- ii. Click **Switch**.

Run queries

To run PostgreSQL queries, select the appropriate database and schema in the dropdown selector.

1. Select the desired database and schema in the dropdown selector next to **Run all**.
2. View databases, schemas, tables, and columns in **PostgreSQL Databases** .
3. You can fill in a table reference from the **Schema Browser** by clicking **>>** next to the desired table.
4. Write a PostgreSQL query, and click **Run all**.

Supported functionality and limitations

1. The [query access control list \(ACL\)](#) permissions model applies:
 - i. The execution happens with the Databricks identity of the user who clicks **Run**.
 - ii. Anyone with at least CAN VIEW access on the query can see the results as they appear in the SQL Editor.
2. Connection-scoped objects, such as temporary tables and transactions, work in a single cell. They do not work across different **Run** clicks, as each is its own session.
3. If `$$` blocks contain statements that have semicolons, only the results for the last statement are shown.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 22, 2026**

Use a notebook to access a database instance

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page contains code examples that show you how to access your Lakebase database instance through [Databricks notebooks](#) and run queries using Python and Scala.

The examples cover different connection strategies to suit different use cases:

- **Single connection:** Used for simple scripts where a single database connection is opened, used, and closed.
- **Connection pool:** Used for high-concurrency workloads, where a pool of reusable connections is maintained.
- **Rotating M2M OAuth token:** Uses short-lived, automatically refreshed OAuth tokens for authentication.

The following examples programmatically generate secure credentials. Avoid directly putting credentials in a notebook. Databricks recommends using one of the following secure methods:

- Store Postgres passwords in Databricks secrets.

 Ask Genie

- Generate OAuth tokens using [M2M OAuth](#).

Before you begin

Ensure you meet the following requirements before accessing your database instance:

- You have a corresponding Postgres role to log in to the database instance. See [Manage PostgreSQL roles](#).
- Your Postgres role is granted the necessary permissions to access the database, schema, or table.
- You can authenticate to the database instance. If you must manually obtain an OAuth token for your database instance, see [Authenticate to a database instance](#).

WARNING

If you're using private link, you need to use a single user cluster.

Python

The Databricks Python SDK can be used to obtain an OAuth token for a respective database instance.

Connect to your database instance from a Databricks notebook using the following Python libraries:

- `psycopg` (`psycopg3`)
- `psycopg` with connection pooling
- `SQLAlchemy`

Prerequisites

Before running the following code examples, upgrade the Databricks SDK for Python to version 0.61.0 or above, then restart Python.

Python

 Ask Genie

```
%pip install databricks-sdk>=0.61.0
%restart_python
```

psycopg

The code examples demonstrate a single connection and the use of a connection pool. For more on how to obtain the database instance and credentials programmatically, see [how to obtain an OAuth token using the Python SDK](#).

Python

```
%pip install "psycopg[binary,pool]"
```

Single connection Connection pool

Python

```
import psycopg

from databricks.sdk import WorkspaceClient
import uuid

w = WorkspaceClient()

instance_name = "<YOUR INSTANCE>"

instance = w.database.get_database_instance(name=instance_name)
cred =
w.database.generate_database_credential(request_id=str(uuid.uuid4()),
instance_names=[instance_name])

# Connection parameters
conn = psycopg.connect(
    host = instance.read_write_dns,
    dbname = "databricks_postgres",
    user = "<YOUR USER>",
    password = cred.token,
    sslmode = "require"
)

# Execute query
```

```

with conn.cursor() as cur:
    cur.execute("SELECT version()")
    version = cur.fetchone()[0]
    print(version)
conn.close()

```

psycopg3

The code example demonstrates the use of a connection pool with a rotating M2M OAuth. It uses `generate_database_credential()`. For more on how to obtain the database instance and credentials programmatically, see [how to obtain an OAuth token using the Python SDK](#).

Python

```
%pip install "psycopg[binary,pool]"
```

Python

```

from databricks.sdk import WorkspaceClient
import uuid

import psycopg
import string
from psycopg_pool import ConnectionPool

w = WorkspaceClient()

class CustomConnection(psycopg.Connection):
    global w
    def __init__(self, *args, **kwargs):
        # Call the parent class constructor
        super().__init__(*args, **kwargs)

    @classmethod
    def connect(cls, conninfo='', **kwargs):
        # Append the new password to kwargs
        cred =
w.database.generate_database_credential(request_id=str(uuid.uuid4()),
instance_names=[instance_name])
        kwargs['password'] = cred.token

```

 Ask Genie


```

# Call the superclass's connect method with updated kwargs
return super().connect(conninfo, **kwargs)

username = "<YOUR USER>"
instance_name = "<YOUR INSTANCE>"
instance = w.database.get_database_instance(name=instance_name)
host = instance.read_write_dns
port = 5432
database = "databricks_postgres"

pool = ConnectionPool(
    conninfo=f"dbname={database} user={username} host={host}",
    connection_class=CustomConnection,
    min_size=1,
    max_size=10,
    open=True
)

with pool.connection() as conn:
    with conn.cursor() as cursor:
        cursor.execute("SELECT version()")
        for record in cursor:
            print(record)

```

SQLAlchemy

The code examples demonstrate a single connection and the use of a connection pool with a rotating M2M OAuth token. For more on how to obtain the database instance and credentials programmatically, see [how to obtain an OAuth token using the Python SDK](#).

Single connection

Connection pool & rotating M2M OAuth

Python

```
%pip install "sqlalchemy>=2.0" "psycopg[binary]"
```

Python

```

from sqlalchemy import create_engine, text

from databricks.sdk import WorkspaceClient
import uuid

w = WorkspaceClient()

instance_name = "<YOUR INSTANCE>"

instance = w.database.get_database_instance(name=instance_name)
cred =
w.database.generate_database_credential(request_id=str(uuid.uuid4()),
instance_names=[instance_name])

user = "<YOUR USER>"
host = instance.read_write_dns
port = 5432
database = "databricks_postgres"
password = cred.token

connection_pool = create_engine(f"postgresql+psycopg://{user}:
{password}@{host}:{port}/{database}?sslmode=require")

with connection_pool.connect() as conn:
    result = conn.execute(text("SELECT version()"))
    for row in result:
        print(f"Connected to PostgreSQL database. Version: {row}")

```

Scala

The code examples show how to programmatically obtain the database instance and credentials, and how to connect to a database instance using a single connection or a connection pool.

Step 1: Use the Databricks Java SDK to obtain an OAuth token

For details on how to obtain the database instance and credentials programmatically, see [how to obtain an OAuth token using the Java SDK](#).

Step 2: Connect to a database instance



Scala

```
import java.sql.{Connection, DriverManager, ResultSet, Statement}

Class.forName("org.postgresql.Driver")

val user = "<YOUR USER>"
val host = instance.getName()
val port = "5432"
val database = "databricks_postgres"
val password = cred.getToken()

val url = f"jdbc:postgresql://{host}:{port}/{database}"

val connection = DriverManager.getConnection(url, user, password)
println("Connected to PostgreSQL database!")

val statement = connection.createStatement()
val resultSet = statement.executeQuery("SELECT version()")

if (resultSet.next()) {
  val version = resultSet.getString(1)
  println(s"PostgreSQL version: $version")
}
```

Is this page

as this page helpful?

Last updated on **Mar 11, 2026**

Access a database instance from SQL clients

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page describes how to access a Lakebase database instance from SQL clients and walks through how to connect to a database instance from `psql`.

You can access a database instance from external tools such as `psql`, DBeaver, and pgAdmin4. These tools also allow you to build dashboards to visualize database metrics for performance, storage usage, and query efficiency.

The `psql` command-line tool is PostgreSQL's built-in client that provides direct, interactive access to your database instance. Using `psql` gives you full Postgres functionality and integrates with existing Postgres workflows and tools.

DBeaver and pgAdmin are open source tools that allow you to manage your Postgres database. For instructions on how to install and connect, see [the DBeaver documentation](#) and [the pgAdmin documentation](#).

Before you begin

- You must install `psql`. For more details, see the [PostgreSQL tools and installation guide](#). `psql` version 14 or higher is required.
- You must install the `libpq` library.
- You have a corresponding Postgres role for your Databricks identity to log in to the database instance. See [Manage PostgreSQL roles](#).
- Your Postgres role is granted the necessary permissions to access the database, schema, or table.
- You can authenticate to the database instance. If you must obtain an OAuth token for your database instance, see [Authenticate to a database instance](#).

Identity and database roles considerations

- The default owner of the instance is the Databricks user or service principal who provisions the database instance.
- During provisioning, a corresponding Postgres role name is automatically created for the identity:
 - **Databricks user:** The Postgres role name is the user's email address.
 - **Databricks Service Principal:** The Postgres role name is the application ID of the service principal.

Connect with `psql`

After your database instance's status shows **Available**, connect to it using the `psql` command-line tool. Copy the connection command from the database instance configuration page.

1. Click  **Apps** in the top right corner and select **Lakebase Postgres**.
2. Click **Provisioned** to open the **Provisioned instances** page.
3. Select the database instance you want to access from `psql`.
4. Click **Connection details**.
5. Copy the **Connect with PSQL** string.
6. Paste the command into your terminal and run it to connect to the database instance using `psql`.

Is this page
as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Feb 4, 2026**

PostgreSQL compatibility

📌 INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see [Region availability](#). For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see [Lakebase Autoscaling](#).

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see [Autoscaling by default](#).

This page describes how a Lakebase database instance is compatible with Postgres. As a managed Postgres service, there are some differences and functionality limitations.

📌 IMPORTANT

Databricks database instances only support Postgres 16.

Optimize queries

The `pg_stat_statements` extension provides detailed information on query performance to help identify slow queries and optimize them.

Users can see the query texts of their queries and any roles from which they inherit privileges. If you have the `ADMIN` privilege on a role and you cannot see the query texts for that role, grant yourself membership in that role. If you have no privileges on a role, you cannot see its query texts for security reasons.

Postgres parameter settings

As a managed Postgres service, the database parameters are set based on the instance size. See all the parameter settings using:

PostgreSQL

```
SHOW ALL;
```

You can configure parameters that have a `user` context at the session, database, or role level. You cannot configure parameters at the instance level.

- Show parameters that can be set at the session, database, or role level.

PostgreSQL

```
SELECT name
FROM pg_settings
WHERE context = 'user';
```

- Set a parameter for the session.

PostgreSQL

```
SET maintenance_work_mem='1 GB';
```

- Set a parameter for all sessions connected to a database

PostgreSQL

```
ALTER DATABASE databricks_postgres SET maintenance_work_mem='1 GB';
```

- Set a parameter for all sessions from a given user.

PostgreSQL


```
ALTER USER "john@company.com" SET maintenance_work_mem='1 GB';
```

Supported encoding and collation

By default, the C.UTF-8 collation is used. C.UTF-8 supports the full range of UTF-8 encoded characters.

The UTF8 encoding (Unicode, 8-bit variable-width encoding) is supported.

To view the encoding and collation for your database, run the following query.

PostgreSQL

```
SELECT
    pg_database.datname AS database_name,
    pg_encoding_to_char(pg_database.encoding) AS encoding,
    pg_database.datcollate AS collation,
    pg_database.datctype AS ctype
FROM
    pg_database
WHERE
    pg_database.datname = 'your_database_name';
```

NOTE

In Postgres, you can't change a database's encoding or collation after it has been created.

Functionality limitations

Roles and permissions

- You can't access the host operating system.
- You can't connect using Postgres `superuser`.
 - Any functionality that requires `superuser` privileges or direct local file system access is not allowed.

- The `databricks_superuser` takes the place of the Postgres `superuser` role. For information about the privileges associated with this role, see [Pre-created roles and permissions](#).

Replication

Creating replication slots, subscriptions, or publications is not supported.

Event triggers

Postgres [event triggers](#) are not supported.

Supported extensions

Extension	Version
address_standardizer	3.3.3
address_standardizer_data_us	3.3.3
autoinc	1
bloom	1
btree_gin	1.3
btree_gist	1.7
citext	1.6
cube	1.5
dict_int	1
earthdistance	1.2
 Ask Genie	

Extension	Version
fuzzystmatch	1.2
hll	2.18
hstore	1.8
insert_username	1
intagg	1.1
intarray	1.5
isn	1.2
lo	1.1
ltree	1.2
moddatetime	1
pg_graphql	1.5.9
pg_jsonschema	0.3.3
pg_prewarm	1.2
pg_stat_statements	1.10
pg_trgm	1.6
pgcrypto	1.3
pgrowlocks	1.2
plpgsql	1

Extension	Version
postgis	3.3.3
postgis_raster	3.3.3
postgis_sfcgal	3.3.3
postgis_tiger_geocoder	3.3.3
postgis_topology	3.3.3
refint	1
seg	1.4
tablefunc	1
tcn	1
tsm_system_rows	1
tsm_system_time	1
unaccent	1.1
uuid-oss	1.1
vector	0.8.0
xml2	1.1